

shravclock

Engineering Report and Product-Oriented Build Documentation

April 12, 2026

Repository: <https://github.com/shravansheth/shravclock>

1 Document Purpose

This document provides a comprehensive technical description of the **shravclock** desk clock firmware project. The report is written as personal build documentation in a polished engineering style, with enough structure to also function as an internal product dossier. It describes the current implementation in detail, records the technical rationale visible in the repository, and separates future work from presently implemented behavior.

Private deployment details such as Wi-Fi credentials, exact installation location, city name, and geographic coordinates are intentionally excluded.

2 Project Summary

shravclock is a battery-powered e-paper desk clock built around a Seeed XIAO ESP32-C6 microcontroller, a 4.2-inch monochrome e-ink display, and a DS3231 real-time clock (RTC). The product goal is not merely to display the current time, but to do so in a way that is visually refined, power-conscious, and robust across long unattended operation.

The firmware combines four major capabilities:

1. persistent and accurate timekeeping via a hardware RTC,
2. low-power minute-based wake, render, and deep-sleep operation,
3. outdoor weather and location-aware presentation over Wi-Fi,
4. a self-hosted configuration portal that avoids reflashing for normal setup changes.

In practical terms, the device behaves like a quiet ambient object. It wakes shortly before each minute boundary, optionally performs an hourly network update, redraws the e-paper display with either a partial or full refresh, and then returns to deep sleep.

3 System Scope

3.1 Primary Functions

The current firmware is designed to provide the following user-visible functions:

- local time display in 24-hour HH:MM format,

- full weekday and calendar date display,
- current outdoor weather with iconography and daily low/high temperature,
- city name display derived from reverse geocoding,
- dark mode display inversion,
- browser-based reconfiguration through a captive setup portal.

3.2 Non-Goals in the Current Revision

The repository indicates several capabilities that are not yet part of the current production firmware:

- active battery measurement and on-screen battery indication,
- low-battery shutdown or special dead-battery display flow,
- adaptive nighttime refresh reduction,
- OTA firmware updates,
- additional sensors beyond the existing RTC and display stack.

4 Hardware Platform

4.1 Core Components

Component	Description
Microcontroller	Seeed XIAO ESP32-C6
Display	WeAct 4.2-inch SPI e-ink panel, SSD1683 controller, black and white, 400 × 300 pixels
RTC	DS3231 Mini on I ² C
User input	Single momentary pushbutton using active-low input with internal pull-up
Battery	3.7 V Li-ion pack, nominally 5000 mAh in the design notes

4.2 Pin Assignment

Signal	GPIO
SPI MOSI	18
SPI SCK	19
E-ink CS	1
E-ink DC	2
E-ink RST	21
E-ink BUSY	16
RTC SDA	22
RTC SCL	23

4.3 Hardware Observations

The hardware selection is internally coherent for a low-power always-visible clock:

- the DS3231 provides reliable time retention independent of Wi-Fi availability,
- the SSD1683 e-paper panel supports partial refresh for low-disturbance minute updates,
- the ESP32-C6 provides Wi-Fi connectivity for time sync, weather fetches, and configuration hosting.

Labeled inference. The use of a dedicated RTC, together with aggressive deep sleep, indicates that power autonomy and display stability are primary design goals rather than secondary optimizations.

5 Software Stack

5.1 Build Environment

The repository is organized as a PlatformIO project using the Arduino framework. The principal firmware implementation resides in `src/main.cpp`.

Item	Details
Build system	PlatformIO
Framework	Arduino
Target board	seeed_xiao_esp32c6
ESP32 platform package	platform-espressif32 via PlatformIO package URL
Partitions	min_spiffs.csv for maximizing application flash budget

5.2 Key Firmware Dependencies

Library or subsystem	Role
GxEPD2	E-paper display driver and rendering support
RTCLib	DS3231 access and RTC time conversion utilities
ArduinoJson v7	JSON parsing for API responses
WiFi / HTTPClient / WiFiClientSecure	network access, HTTP weather fetch, HTTPS geocoding
Preferences	non-volatile configuration storage through ESP32 NVS
WebServer + DNSServer	captive setup portal implementation

6 Repository Structure

The repository is intentionally compact. The most important files in the current revision are:

Path	Purpose
src/main.cpp	Main firmware implementation and runtime state machine
FIRMWARE_OVERVIEW.md	High-level architecture and design notes
platformio.ini	PlatformIO environment and dependency definitions
include/WeatherStationFonts.h	Meteocons bitmap font asset used for weather icons
battery-measuring/battery_code_backup.cpp	battery-measurement feature code not yet integrated
README.md	short project summary

7 Functional Specification

7.1 Timekeeping

The displayed time is formatted in 24-hour style and rendered without seconds. Time is stored in UTC on the DS3231 RTC. Local presentation is produced by applying a POSIX timezone string at display time.

This is an important architectural decision:

- the RTC remains timezone-agnostic,
- configuration changes to timezone rules can be applied without rewriting the conceptual time basis,
- daylight saving transitions are handled through timezone conversion logic rather than through local-time RTC storage.

On cold boot, the firmware connects to Wi-Fi, performs NTP synchronization, and adjusts the DS3231 using the synchronized UTC epoch.

7.2 Date Presentation

The date area contains two centered lines:

- full weekday name,
- abbreviated month, day of month, and year.

The implementation computes text bounds dynamically before drawing, allowing the firmware to center each string without hard-coded assumptions about pixel width.

7.3 Weather Presentation

Current weather is sourced from Open-Meteo and shown as:

- current temperature in degrees Fahrenheit,
- weather icon selected from WMO weather code,
- daily low and high temperatures,
- city label beneath the weather line.

Weather data is not fetched every minute. Instead, the device refreshes weather once per hour near the top of the hour, with retry logic on failure and persistent in-memory caching across deep sleep cycles.

7.4 Weather Iconography

The icon system uses a custom renderer for the Meteocons bitmap font format included in the repository. The firmware maps WMO weather codes to a reduced symbolic visual language such as clear, partly cloudy, fog, rain, snow, showers, and thunderstorm.

This approach provides a compact and deterministic icon pipeline without external graphical assets beyond the bundled font table.

7.5 City Name Resolution

The city name is obtained through reverse geocoding from Nominatim using the configured latitude and longitude. The firmware prioritizes **city**, then **town**, then **village**, and finally falls back to a generic display name if the more specific fields are absent.

The resulting city label is cached in RTC memory so it can persist across deep sleep cycles without repeated network access.

7.6 Dark Mode

Dark mode reverses the display polarity from black-on-white to white-on-black and adds a crescent moon overlay in the top-left area of the screen. The mode is user-configurable through the setup portal and is stored in NVS so that it survives resets and sleep cycles.

8 Display Layout and Typography

8.1 Layout

The 400×300 pixel display is arranged as a vertically centered composition:

- top emphasis on time,
- mid-screen date information,
- lower weather block including icon, temperature, low/high values, and city name.

The layout is computed programmatically. Bounding boxes are measured at runtime and used to center or align content precisely. This makes the UI resilient to font metrics and reduces the need for manual per-string adjustments.

8.2 Typography

The firmware uses a small set of Adafruit GFX fonts with clear role separation:

- bold sans serif at double scale for the time,
- bold sans serif for weekday,
- regular sans serif for date,
- bold larger sans serif for current temperature,
- smaller sans serif for low/high and city.

The degree marker is drawn manually as a circle instead of relying on a font glyph. This gives consistent visual positioning in the weather block.

Labeled inference. The visual design aims for a restrained instrument-panel aesthetic rather than a decorative smart display aesthetic. The choice of typography, centered composition, and sparse iconography supports that reading.

9 Power and Sleep Architecture

9.1 Setup-Driven Runtime Model

The firmware is effectively `setup()`-driven. The device wakes, restores state, performs any required work, draws the screen, and returns to deep sleep. The `loop()` function is not used for continuous normal operation and simply routes back into sleep.

9.2 Wake Timing

The device wakes at approximately 58 seconds past each minute. This creates a short pre-minute execution window in which the firmware can:

- check for button hold requests,
- perform an hourly weather update if needed,
- defer display initialization until the appropriate render moment.

This scheme allows the visible display update to remain aligned with the minute boundary while still giving the processor enough time to prepare.

9.3 Full Versus Partial Refresh

The refresh strategy is:

- full refresh on cold boot,
- full refresh at the top of each hour,
- partial time-only refresh on other minute transitions.

The time refresh region is precomputed and aligned to 8-pixel boundaries, which matches the partial window constraints of the display controller.

9.4 Display Initialization Deferral

One of the most technically significant design choices is that display initialization is deferred during timer wakes. The repository documents that early initialization would pulse the display reset line and cause a visible artifact at the `:58` wake moment. Deferring `display.init()` until just before drawing prevents an unnecessary visible flash and makes the wake cycle effectively silent from the user perspective.

9.5 GPIO Hold Strategy

Before deep sleep, the firmware enables GPIO hold on the display control lines associated with reset and signaling. This prevents the ESP32-C6 boot ROM from briefly glitching those lines during the next wake sequence, which would otherwise wake the e-paper controller and create a visible flash.

This is a thoughtful hardware-software integration detail and represents one of the strongest technical aspects of the project.

10 State Management

10.1 Persistent Configuration

Long-lived user configuration is stored in ESP32 NVS under a dedicated namespace. The stored settings include:

- Wi-Fi SSID,
- Wi-Fi password,
- POSIX timezone string,
- corresponding IANA timezone name,
- latitude,
- longitude,
- dark mode flag.

10.2 RTC Memory State

Transient but sleep-persistent runtime state is stored in RTC memory using `RTC_DATA_ATTR`. This includes:

- weather validity and most recently fetched values,
- last successful weather fetch timestamp,
- dark mode working state,
- cached city name,
- validity flag for the RTC-backed state block.

This architecture reduces network activity and improves resilience across minute wake cycles.

11 Networked Features

11.1 NTP Synchronization

On cold boot the device performs NTP synchronization using public time servers. Before SNTP is started, the firmware explicitly resets the ESP32 system clock to zero. The repository notes that without doing so, stale system time retained in RTC slow memory could be misinterpreted as a fresh NTP result.

This is a subtle but important correctness measure.

11.2 Weather API

Weather data is requested from Open-Meteo over plain HTTP. The project notes explain that HTTPS was avoided for this specific endpoint because the ESP32-C6 TLS stack exhibited handshake reliability issues against the service configuration, while the weather data is public and unauthenticated.

From a product perspective, this is a pragmatic engineering tradeoff:

- reliability of fetches was prioritized,
- no secrets are exposed by the request itself,
- the endpoint provides non-sensitive public data.

11.3 Geocoding

Reverse geocoding and setup-portal city search use Nominatim over HTTPS, with a custom user agent string and relaxed certificate verification through `setInsecure()`.

Labeled inference. For a personal desk device, this is a reasonable integration choice, though a more security-constrained product version would likely replace permissive TLS verification with pinned or properly validated certificates.

12 Configuration Portal

12.1 Entry Method

The setup portal is entered by holding the button for approximately one second while the processor is awake. The current design does not support true deep-sleep button wake on the chosen input pin because GPIO 17 is not a low-power wake-capable pin on the ESP32-C6.

12.2 Portal Behavior

When activated, the firmware:

- draws a setup screen on the e-paper display,
- starts a local Wi-Fi access point,
- enables a DNS catch-all responder for captive portal behavior,
- serves a web configuration interface on port 80,
- optionally connects to the previously saved home network in STA mode for city search proxying.

The portal times out after five minutes if no configuration is saved.

12.3 User-Facing Setup Functions

The web UI allows the user to:

- confirm Wi-Fi connectivity state,
- enter or preserve Wi-Fi credentials,
- search for a city and store latitude/longitude,
- choose a timezone from a curated list,
- toggle dark mode,
- save and restart the device.

13 Boot and Execution Flow

13.1 Cold Boot Path

The cold boot path can be summarized as:

1. load saved configuration,
2. initialize hardware interfaces,
3. initialize the display immediately,
4. initialize the RTC,
5. check for a button-held portal request,
6. compute layout geometry,
7. connect to Wi-Fi,
8. synchronize time from NTP to the RTC,
9. fetch city name,
10. fetch weather,
11. render a full screen,
12. enter deep sleep.

13.2 Minute Wake Path

Normal minute wake operation can be summarized as:

1. wake at :58,
2. load configuration and restore sleep-persistent state,
3. release held GPIO lines,
4. check whether setup mode is requested,

5. if appropriate, fetch hourly weather update,
6. wait for the safe draw window,
7. initialize the display only when draw time is reached,
8. perform either full or partial refresh,
9. return to deep sleep.

14 Current Limitations

The current implementation is already functional and coherent, but it includes several known limitations:

- button entry to setup mode is only available during the active wake window,
- battery level is not yet measured or displayed in the production firmware,
- weather refresh cadence is fixed rather than adaptive,
- the portal includes no image assets, diagrams, or richer diagnostics,
- networking is intentionally opportunistic rather than continuously connected.

15 Product Characterization

If treated as a small product rather than only a hobby prototype, the current firmware can be characterized as follows.

15.1 Strengths

- Strong low-power architecture based on deep sleep and RTC ownership of time.
- Careful e-paper behavior management, especially around reset artifacts.
- User-configurable setup without requiring firmware recompilation.
- Clean information hierarchy on screen.
- Limited but purposeful network dependency.

15.2 Risks for Productization

- Field update strategy is not yet present.
- Battery behavior is not yet fully integrated.
- Some external services are public community services and may not be suitable unchanged for scaled deployment.
- TLS verification is permissive for geocoding requests.

16 Build and Development Notes

The repository indicates a standard PlatformIO workflow for local builds and flashing. Development currently appears optimized for direct firmware iteration rather than for a release pipeline with automated validation.

Labeled inference. This is appropriate for the project stage. The codebase reads like a carefully iterated personal device firmware rather than a generalized SDK or mass-production codebase.

17 Planned and Future Work

This section records features that are either explicitly staged in the repository or recommended as natural next steps for the product.

17.1 Planned Battery Measurement Integration

The repository includes a non-integrated battery feature draft in `battery-measuring/battery_code_backup.cpp`. The intended design uses a resistor divider wired to an ADC input so the firmware can estimate pack voltage and derive battery percentage.

The staged implementation includes:

- voltage sampling through the ADC,
- a discharge-curve-based percentage estimate,
- charging-state detection,
- an on-screen battery icon or charging indicator,
- a dedicated low-battery or dead-battery screen,
- extended sleep behavior when the battery is critically low.

This is the most obvious near-term enhancement because much of the logic has already been conceptually specified and partially written. Once the resistor divider is physically wired as required, the feature should integrate naturally into the existing hourly full-refresh flow and low-power architecture.

17.2 Recommended Adaptive Night Refresh Policy

An additional recommended feature is reduced refresh frequency during nighttime hours. The existing firmware updates the time every minute, but e-paper clocks do not necessarily need identical visual update cadence across the full 24-hour day.

A future adaptive policy could:

- retain the current minute refresh rate during daytime,
- switch to a slower cadence during configured night hours,
- continue to preserve hourly weather maintenance if desired,
- reduce battery consumption and unnecessary display activity.

This feature aligns strongly with the current design philosophy because it improves power efficiency without changing the visual identity or core use model of the device.

17.3 Other Viable Future Enhancements

Other future additions that would fit the product well include:

- alternate visual themes or layout modes,
- indoor environmental sensing such as room temperature,
- OTA update support,
- sunrise, sunset, or moon-phase indicators,
- lightweight calendar or reminder overlays.

18 Conclusion

`shravclock` is a thoughtfully engineered personal desk clock with a notably strong low-power and e-paper-oriented architecture. Its most mature qualities are the wake-refresh-sleep execution model, the deliberate handling of display hardware quirks, the RTC-centered time strategy, and the inclusion of a self-contained configuration portal.

At its current stage, the project already operates as a complete and polished personal device. The most natural next milestone is battery instrumentation, followed by adaptive nighttime behavior and broader product-hardening improvements. As documented in this report, the project is already substantially beyond a simple clock build; it is a compact embedded product with clear architecture, deliberate tradeoffs, and a credible path toward a more fully featured standalone object.